

Java: Un Análisis Teórico, Histórico y Técnico de su Arquitectura y Máquina Virtual

Parte I: Fundamentos Filosóficos y Orígenes Históricos

Para comprender la esencia del lenguaje de programación Java, es imperativo analizar no solo su sintaxis y semántica, sino también el contexto histórico y las presiones filosóficas que forjaron su creación. Java no surgió en un vacío tecnológico; fue una respuesta deliberada y calculada a los desafíos de la ingeniería de software de principios de la década de 1990. Su diseño representa una refutación de ciertos paradigmas preexistentes y una apuesta por un futuro de computación distribuida y heterogénea. Esta sección explora el "porqué" de Java, desde sus orígenes en un proyecto de electrónica de consumo hasta su consagración como el lenguaje de la World Wide Web, desentrañando la filosofía que lo convirtió en una fuerza transformadora en la industria.

El Nacimiento de un Lenguaje Universal: Del Proyecto "Oak" a la Conquista de la Web

A principios de la década de 1990, el panorama del desarrollo de software estaba dominado por lenguajes como C y C++. Si bien eran inmensamente potentes y ofrecían un control de bajo nivel sobre el hardware, presentaban desafíos significativos en términos de portabilidad, gestión de memoria y seguridad. El código escrito en C++ debía ser compilado específicamente para cada arquitectura de hardware y sistema operativo, lo que dificultaba el desarrollo de software para un mercado cada vez más diverso. La gestión manual de la memoria era una fuente constante de errores complejos, como fugas de memoria y punteros corruptos. En este contexto, en 1991, un pequeño equipo de ingenieros de Sun Microsystems, conocido como el "Green Team" y liderado por James Gosling, inició un proyecto de investigación. Su objetivo inicial no era la computación de escritorio ni Internet, sino el emergente mercado de la electrónica de consumo y la televisión por cable digital. El proyecto, cuyo lenguaje de programación se denominó internamente "Oak" (en honor a un roble que Gosling veía desde su ventana), buscaba crear una plataforma de software que fuera flexible, portátil y segura, capaz de ejecutarse en una nueva generación de dispositivos inteligentes como decodificadores de televisión y electrodomésticos. La principal motivación era superar el problema de la enorme diversidad de microprocesadores y controladores electrónicos de bajo coste, que variaban constantemente y utilizaban diferentes conjuntos de instrucciones. Sin embargo, el proyecto "Oak" no encontró el éxito esperado en el mercado de la electrónica de consumo. Simultáneamente, el mundo estaba siendo testigo de la explosión de la World Wide Web. El equipo del Green Team reconoció una oportunidad única: la arquitectura que habían diseñado para la portabilidad en dispositivos embebidos heterogéneos era, por una afortunada coincidencia, la solución perfecta para el entorno inherentemente heterogéneo de la

web, un ecosistema de sistemas operativos, arquitecturas de hardware y navegadores dispares. El problema de la "diversidad de hardware" en los dispositivos inteligentes era análogo al problema de la "diversidad de plataformas" en Internet. Este pivote estratégico no fue una reinención, sino la aplicación de una solución ya existente a un problema de una escala y un impacto mucho mayores.

En 1995, Sun Microsystems relanzó oficialmente el lenguaje bajo el nuevo nombre de "Java". El nombre fue elegido por Gosling, según la anécdota, mientras tomaba una taza de café, en honor a la isla de Indonesia famosa por su producción cafetera, lo que también inspiró el icónico logotipo de la taza de café caliente. El punto de inflexión para la adopción masiva de Java fue su integración con Netscape Navigator, el navegador web más popular de la época. Esta integración permitió la ejecución de pequeños programas interactivos, conocidos como "applets", directamente dentro de una página web, una capacidad revolucionaria que transformó la web estática en una plataforma dinámica e interactiva.

La Promesa de la Portabilidad: Un Análisis Profundo de la Filosofía "Write Once, Run Anywhere" (WORA)

La propuesta de valor central de Java, y el principio que lo diferenció radicalmente de sus predecesores, quedó inmortalizada en su lema: "Write Once, Run Anywhere" (WORA), que se traduce como "Escríbelo una vez, ejecútalo en cualquier lugar". Esta filosofía prometía liberar a los desarrolladores de la tediosa y costosa tarea de adaptar y recompilar su código para cada plataforma de destino. Con Java, un programa podía ser escrito una sola vez y ejecutarse sin modificaciones en sistemas Windows, Unix, macOS o cualquier otra plataforma compatible. El habilitador técnico fundamental de esta promesa fue la introducción de la Máquina Virtual de Java (JVM, por sus siglas en inglés). La JVM es una capa de abstracción de software que se interpone entre el código del programa y el sistema operativo y hardware subyacentes. A diferencia de C++, que compila el código fuente directamente a código máquina nativo y específico de una plataforma, el compilador de Java traduce el código fuente a un formato intermedio e independiente de la arquitectura llamado *bytecode*. Este bytecode puede ser ejecutado por cualquier dispositivo que disponga de una implementación compatible de la JVM, que se encarga de traducir las instrucciones del bytecode al código máquina nativo del sistema anfitrión en tiempo de ejecución. Por lo tanto, la palabra "Java" se refiere a dos conceptos inseparables: el lenguaje de programación y la Máquina Virtual que ejecuta sus programas. Esta arquitectura fue el resultado directo de los cinco objetivos principales que guiaron el diseño del lenguaje :

1. **Debería usar el paradigma de la programación orientada a objetos.** Este enfoque, que ya era una tendencia clara, facilitaba la creación de software modular y reutilizable.
2. **Debería permitir la ejecución de un mismo programa en múltiples sistemas operativos.** Este es el núcleo de la filosofía WORA, logrado a través de la JVM.
3. **Debería incluir soporte integrado para redes de computadoras.** El diseño de Java anticipó la importancia de la computación distribuida e Internet.
4. **Debería permitir la ejecución segura de código desde fuentes remotas.** La seguridad era primordial, especialmente para la ejecución de applets descargados de la web.
5. **Debería ser fácil de aprender y usar.** Para ello, se buscó una sintaxis familiar para los programadores de C/C++ pero eliminando las características más complejas y propensas a errores de dichos lenguajes.

En Respuesta a la Complejidad: Cómo Java Abordó las Limitaciones de C++

La identidad fundamental de Java no puede entenderse sin C++. Casi todas sus características de diseño más distintivas son una reacción directa y deliberada a los problemas y la complejidad percibidos en C++. James Gosling, en sus ratos libres, había comenzado a crear un lenguaje que intentaba solucionar los fallos que encontraba en C++. Por lo tanto, Java no es simplemente una evolución, sino una refutación filosófica del enfoque de C++, priorizando la seguridad y la productividad del desarrollador sobre el control de bajo nivel absoluto.

Gestión de Memoria

La diferencia más significativa radica en la gestión de la memoria. En C++, el programador es responsable de asignar memoria dinámicamente con el operador `new` y de liberarla explícitamente con `delete`. Este control manual, aunque potente, es una de las principales fuentes de errores en C++, como las fugas de memoria (olvidar liberar memoria) y los punteros colgantes (intentar usar memoria ya liberada). Java abordó este problema de raíz introduciendo la gestión automática de la memoria a través de un componente de la JVM conocido como el **Recolector de Basura (Garbage Collector)**. El recolector de basura monitoriza los objetos en memoria y libera automáticamente aquellos que ya no son referenciados por el programa, eliminando una clase entera de errores y liberando al programador de esta compleja tarea. Esta decisión fue una causa directa de la búsqueda de mayor robustez y simplicidad.

Punteros y Seguridad

Estrechamente relacionado con la gestión de la memoria está el concepto de punteros. C++ permite la aritmética de punteros, que da al programador la capacidad de manipular directamente direcciones de memoria. Si bien esto es esencial para la programación de sistemas de bajo nivel, también es una puerta de entrada a vulnerabilidades de seguridad y errores difíciles de depurar, una condición conocida como "memory unsafety". Java eliminó por completo la aritmética de punteros. En su lugar, utiliza un modelo de referencias gestionadas. Los programadores manipulan referencias a objetos, pero no pueden acceder ni modificar las direcciones de memoria subyacentes. Esta abstracción, junto con otras políticas de seguridad, hace que sea mucho más difícil escribir código malicioso o propenso a errores de memoria en Java.

Comportamiento Indefinido vs. Robustez

El estándar de C++ deja ciertas situaciones como "comportamiento indefinido", lo que significa que el compilador y el sistema de ejecución son libres de hacer cualquier cosa. Ejemplos comunes incluyen el acceso a un array fuera de sus límites, la desreferencia de un puntero nulo o el desbordamiento de un entero con signo. En el mejor de los casos, el programa se bloquea; en el peor, continúa ejecutándose con datos corruptos, provocando errores sutiles y difíciles de diagnosticar. Java adoptó un enfoque opuesto, priorizando la robustez y la previsibilidad. En estas mismas situaciones, Java impone verificaciones en tiempo de ejecución que lanzan excepciones bien definidas y manejables, como `ArrayIndexOutOfBoundsException` o

NullPointerException. Este comportamiento hace que los programas Java sean más fáciles de depurar y más fiables en producción.

Independencia de la Plataforma y Consistencia

Como se mencionó anteriormente, la diferencia fundamental en el modelo de compilación (código nativo para C++ vs. bytecode para Java) es la base de la portabilidad de Java. Esta diferencia también tiene implicaciones en la consistencia del lenguaje. Por ejemplo, el tamaño de un tipo de dato primitivo como int en C++ puede variar según la plataforma (por ejemplo, 16 o 32 bits), lo que puede llevar a comportamientos inesperados al portar el código. En Java, los tamaños de todos los tipos de datos primitivos están estrictamente definidos en la especificación del lenguaje (int es siempre de 32 bits), garantizando un comportamiento consistente en cualquier plataforma.

Simplicidad del Lenguaje

Para reducir la complejidad y la curva de aprendizaje, los diseñadores de Java omitieron deliberadamente varias características de C++ consideradas complejas o problemáticas. La herencia múltiple, que puede conducir a jerarquías de clases ambiguas y frágiles (el "problema del diamante"), fue reemplazada por la implementación de múltiples interfaces, un modelo más simple y robusto. También se eliminaron la sobrecarga de operadores y las plantillas (templates), que fueron sustituidas por una implementación más simple de genéricos, aunque con diferencias técnicas importantes en su funcionamiento interno.

Parte II: La Arquitectura de la Plataforma y su Modelo de Ejecución

La filosofía de Java se materializa en una arquitectura de plataforma cuidadosamente diseñada. Esta arquitectura no solo habilita la portabilidad, sino que también define una clara separación de responsabilidades entre las herramientas de desarrollo y los entornos de ejecución. Comprender esta estructura es fundamental para entender cómo el código Java pasa de ser un simple archivo de texto a un programa en ejecución. Esta sección desglosa los componentes clave del ecosistema Java y traza el ciclo de vida completo del código, revelando el ingenioso modelo híbrido que combina compilación e interpretación para lograr un equilibrio entre portabilidad y rendimiento.

El Ecosistema Trinitario: Desmitificando el Kit de Desarrollo (JDK), el Entorno de Ejecución (JRE) y la Máquina Virtual (JVM)

La plataforma Java está compuesta por tres componentes principales, a menudo fuente de confusión para los principiantes: el Kit de Desarrollo de Java (JDK), el Entorno de Ejecución de Java (JRE) y la Máquina Virtual de Java (JVM). Estos componentes forman una jerarquía donde cada uno es un superconjunto del siguiente.

Java Virtual Machine (JVM)

La JVM es el corazón y el alma de la plataforma Java. Es una especificación para una máquina

de computación abstracta que permite a un ordenador ejecutar programas Java. No es un componente físico, sino un software que simula una máquina y reside en el sistema operativo real. Sus responsabilidades principales son:

- **Proporcionar un entorno de ejecución:** Crea un espacio aislado para que las aplicaciones Java se ejecuten de forma segura.
- **Interpretar y compilar el bytecode:** Convierte el bytecode (independiente de la plataforma) en instrucciones de código máquina nativo que el procesador del sistema anfitrión puede entender.
- **Gestionar la memoria:** Realiza la asignación de memoria y la recolección de basura de forma automática.
- **Garantizar la independencia de la plataforma:** Al abstraer las diferencias del hardware y del sistema operativo subyacente, la JVM es el componente que hace realidad la promesa "Write Once, Run Anywhere".

Java Runtime Environment (JRE)

El JRE es el paquete de software necesario para *ejecutar* una aplicación Java ya compilada. Es un subconjunto del JDK. Un usuario final que solo desea ejecutar un programa Java en su máquina necesita instalar el JRE, pero no el JDK completo. El JRE agrupa dos componentes esenciales:

- **La Máquina Virtual de Java (JVM):** El motor de ejecución descrito anteriormente.
- **Bibliotecas de Clases Estándar (Java API):** Una vasta colección de código preescrito que proporciona funcionalidades comunes, como manipulación de cadenas, operaciones de red, acceso a bases de datos (JDBC), interfaces gráficas (Swing, AWT), etc..

El JRE no incluye herramientas de desarrollo como el compilador o el depurador. Su único propósito es proporcionar la plataforma para la ejecución.

Java Development Kit (JDK)

El JDK es el kit de herramientas completo para *desarrollar* software en Java. Es un superconjunto del JRE, lo que significa que incluye todo lo que contiene el JRE (la JVM y las bibliotecas de clases) y, además, añade las herramientas esenciales para la programación. Los componentes clave exclusivos del JDK incluyen:

- **Compilador (javac):** La herramienta que convierte el código fuente .java en bytecode .class.
- **Depurador (jdb):** Una herramienta para encontrar y corregir errores en el código.
- **Generador de Documentación (javadoc):** Crea documentación de la API en formato HTML a partir de comentarios en el código fuente.
- **Otras utilidades:** Herramientas para archivar (jar), monitorizar (jconsole), y más.

Esta separación arquitectónica entre JDK y JRE no es meramente técnica; refleja una profunda comprensión de los roles en el ciclo de vida del software. El desarrollador, que necesita compilar, depurar y empaquetar, utiliza el JDK. El usuario final o el servidor de producción, que solo necesita ejecutar la aplicación, instala únicamente el JRE. Este diseño permite un despliegue más eficiente, ligero y seguro, ya que los entornos de producción no se cargan con herramientas de desarrollo innecesarias que podrían aumentar la superficie de ataque y el tamaño de la instalación.

Tabla 1: Comparativa de Componentes de la Plataforma Java (JDK vs. JRE vs.

JVM)

Componente	Propósito Principal	Contenido Clave	Usuario Típico
JDK (Java Development Kit)	Desarrollar, compilar y depurar aplicaciones Java.	JRE (JVM + Bibliotecas) + Herramientas de desarrollo (compilador, depurador).	Desarrollador de software.
JRE (Java Runtime Environment)	Ejecutar aplicaciones Java.	JVM + Bibliotecas de clases estándar (Java API).	Usuario final, servidor de producción.
JVM (Java Virtual Machine)	Ejecutar el bytecode de Java.	Motor de ejecución, gestor de memoria, cargador de clases.	Es un componente interno, no se instala por separado.

El Viaje del Código: Fases del Proceso de Compilación y Ejecución en Java

El modelo de ejecución de Java es un proceso de múltiples fases que combina elementos de compilación e interpretación. Este diseño híbrido es un compromiso de ingeniería deliberado para equilibrar la portabilidad y el rendimiento.

- **Fase 1: Escritura del Código Fuente** El proceso comienza cuando un desarrollador escribe el código de la aplicación en archivos de texto plano con la extensión .java. Estos archivos contienen código legible por humanos que sigue la sintaxis y las reglas del lenguaje de programación Java.
- **Fase 2: Compilación a Bytecode** A continuación, se utiliza el compilador de Java, javac, que forma parte del JDK. El compilador toma como entrada uno o más archivos .java y realiza varias tareas:
 1. **Análisis Léxico:** El código fuente se descompone en una secuencia de "tokens" (palabras clave, identificadores, operadores, etc.).
 2. **Análisis Sintáctico (Parsing):** Se comprueba que la secuencia de tokens se ajuste a la gramática del lenguaje Java. Se verifica la sintaxis correcta, como el uso adecuado de puntos y comas, llaves y paréntesis.
 3. **Análisis Semántico y Generación de Código:** Se verifica la coherencia del código (por ejemplo, que los tipos de datos sean compatibles en las asignaciones) y se genera una representación intermedia. El resultado final de este proceso no es código máquina nativo, sino un archivo con la extensión .class por cada clase definida en el código fuente. Este archivo contiene el **bytecode**, un conjunto de instrucciones optimizadas para ser ejecutadas por la JVM. Este bytecode es la clave de la portabilidad de Java, ya que es completamente independiente de la plataforma de hardware o del sistema operativo.
- **Fase 3: Carga y Ejecución por la JVM** Una vez que el programa está compilado en bytecode, está listo para ser ejecutado. Esto se hace utilizando el comando java, que es parte del JRE.
 1. **Inicio de la JVM:** El comando java inicia una instancia de la Máquina Virtual de Java.
 2. **Carga de Clases:** El subsistema de carga de clases (Class Loader) de la JVM localiza el archivo .class de la clase principal especificada y lo carga en la memoria

de la JVM. También carga cualquier otra clase que sea necesaria durante la ejecución.

3. **Verificación del Bytecode:** Antes de ejecutar el código, un verificador de bytecode examina las instrucciones para asegurarse de que son seguras, no violan las restricciones de acceso y no realizan operaciones ilegales. Este es un paso de seguridad crucial.
4. **Ejecución:** Finalmente, el Motor de Ejecución (Execution Engine) de la JVM toma el control. Puede operar de dos maneras:
 - **Interpretación:** Lee el bytecode instrucción por instrucción y lo traduce a código máquina nativo sobre la marcha para su ejecución.
 - **Compilación Just-In-Time (JIT):** Para mejorar el rendimiento, la JVM monitoriza la ejecución del código. Si detecta que un fragmento de código (un método, por ejemplo) se ejecuta con frecuencia, el compilador JIT lo traduce a código máquina nativo y lo almacena en caché. Las futuras llamadas a ese método ejecutarán directamente el código nativo, mucho más rápido que la interpretación.

Este modelo híbrido es la solución al dilema de portabilidad versus rendimiento. La compilación inicial a un formato intermedio (bytecode) es la causa de la independencia de la plataforma, una ventaja típica de los lenguajes interpretados. La compilación tardía y selectiva (JIT) en la máquina de destino es la causa de la optimización del rendimiento, acercándose a la velocidad de los lenguajes compilados nativamente. Es una arquitectura de "compilar una vez, optimizar en todas partes".

Bytecode: El ADN de la Independencia de Plataforma

El bytecode es el lenguaje intermedio que se encuentra en el corazón de la portabilidad de Java. Es un lenguaje de bajo nivel, similar en concepto al lenguaje ensamblador, pero con una diferencia fundamental: no está diseñado para un procesador físico, sino para un procesador virtual, la JVM.

Su estructura consiste en una secuencia de códigos de operación (opcodes), la mayoría de los cuales tienen un solo byte de longitud, lo que da origen al nombre "bytecode". Cada opcode representa una instrucción específica para la JVM, como cargar una variable, realizar una operación aritmética o invocar un método. Estos opcodes pueden ir seguidos de operandos que proporcionan información adicional para la instrucción. Las instrucciones se dividen en varias categorías, incluyendo :

- **Carga y almacenamiento:** Mover valores entre las variables locales y la pila de operandos (ej. `iload_1`, `istore`).
- **Aritmética y lógica:** Realizar operaciones matemáticas y lógicas (ej. `iadd`, `fcmpl`).
- **Control de flujo:** Bifurcaciones condicionales e incondicionales (ej. `ifeq`, `goto`).
- **Invocación de métodos:** Llamar a métodos y devolver valores (ej. `invokespecial`, `areturn`).
- **Manipulación de objetos:** Crear y manipular objetos y sus campos (ej. `new`, `putfield`).

La ejecución del bytecode en la JVM se basa en una arquitectura de pila. La mayoría de las instrucciones toman sus operandos de una estructura de datos llamada "pila de operandos", realizan la operación y dejan el resultado de nuevo en la pila. Para ilustrar esto, consideremos un ejemplo de código Java simple y su bytecode correspondiente :

Código Java:

```
int sum(int a, int b) {
```

```
    return a + b;
}
```

Bytecode (formato javap):

```
0: iload_1      // Carga la variable local en el índice 1 (parámetro
'a') en la pila de operandos.
1: iload_2      // Carga la variable local en el índice 2 (parámetro
'b') en la pila de operandos.
2: iadd         // Toma los dos valores enteros de la cima de la
pila, los suma y empuja el resultado de vuelta a la pila.
3: ireturn      // Devuelve el valor entero que se encuentra en la
cima de la pila como resultado del método.
```

Este ejemplo demuestra la naturaleza compacta y basada en pila del bytecode. La instrucción `iload_1` empuja el valor del primer argumento (a) a la pila. `iload_2` empuja el valor de b. La pila ahora contiene [a, b]. La instrucción `iadd` consume estos dos valores, los suma y empuja el resultado (a+b) de nuevo a la pila. Finalmente, `ireturn` toma este resultado de la pila y finaliza la ejecución del método. Este conjunto de instrucciones, contenido en el archivo `.class`, es universal y puede ser entendido por cualquier JVM, sin importar la plataforma subyacente.

Parte III: Anatomía Detallada de la Máquina Virtual de Java (JVM)

La Máquina Virtual de Java es una obra maestra de la ingeniería de software, un entorno de ejecución sofisticado que gestiona el ciclo de vida completo de una aplicación Java. Para comprender verdaderamente cómo funciona Java, es esencial diseccionar la JVM en sus subsistemas funcionales. Estos componentes trabajan en conjunto para cargar código de forma segura, gestionar la memoria de manera eficiente y ejecutar el bytecode con un rendimiento optimizado. Esta sección profundiza en la arquitectura interna de la JVM, explorando el Subsistema de Carga de Clases, las Áreas de Datos en Tiempo de Ejecución y el Motor de Ejecución.

El Guardián de la Entrada: El Subsistema de Carga de Clases (Class Loader Subsystem)

El Subsistema de Carga de Clases actúa como el guardián de la JVM. Es responsable de encontrar y cargar dinámicamente los archivos de clase (`.class`) en la memoria de la JVM en tiempo de ejecución. Una característica clave es que las clases no se cargan todas al inicio de la aplicación, sino "bajo demanda", es decir, solo cuando son necesarias por primera vez. Este proceso se divide en tres fases principales y estrictamente ordenadas.

Proceso de Carga en Tres Fases

1. **Loading (Carga):** Esta es la primera fase, donde el cargador de clases localiza el archivo `.class` (ya sea en el sistema de archivos local, en una red o en otra fuente), lee sus datos binarios (el bytecode) y crea una representación interna de esa clase dentro del Área de

Métodos de la JVM. Por cada clase cargada, la JVM también crea una instancia de `java.lang.Class` para representarla.

2. **Linking (Enlazado):** Una vez que la clase está cargada, debe ser integrada en el estado de ejecución de la JVM. Este proceso, llamado enlazado, consta de tres sub-pasos:
 - **Verification (Verificación):** Este es un paso de seguridad crucial. El verificador de bytecode examina el código cargado para asegurarse de que es estructuralmente correcto, se adhiere a la semántica del lenguaje Java y no viola las reglas de seguridad de la JVM. Comprueba, por ejemplo, que no haya desbordamientos de pila, que las conversiones de tipo sean válidas y que no se violen las reglas de acceso a miembros privados.
 - **Preparation (Preparación):** En esta etapa, la JVM asigna memoria para todas las variables estáticas (variables de clase) definidas en la clase y las inicializa con sus valores por defecto (por ejemplo, 0 para tipos numéricos, false para booleanos y null para referencias a objetos).
 - **Resolution (Resolución):** El bytecode contiene referencias simbólicas a otras clases, campos y métodos (por ejemplo, por sus nombres). La resolución es el proceso de transformar estas referencias simbólicas en referencias directas o punteros de memoria. Este paso puede ocurrir de forma tardía, justo antes de que se utilice una referencia simbólica por primera vez.
3. **Initialization (Inicialización):** Esta es la fase final, donde se ejecuta el código de inicialización de la clase. Esto incluye la ejecución de los bloques de inicialización estáticos (bloques `static {}`) y la asignación de los valores iniciales especificados por el programador a las variables estáticas. Esta fase se activa la primera vez que se accede activamente a la clase (por ejemplo, al crear una instancia o al llamar a un método estático).

Jerarquía de Class Loaders y el Modelo de Delegación

La JVM no utiliza un único cargador de clases, sino una jerarquía de tres cargadores principales. Este diseño es fundamental para la seguridad y la modularidad de la plataforma.

- **Bootstrap Class Loader:** Es la raíz de la jerarquía y está implementado en código nativo (C/C++). Es responsable de cargar las clases más fundamentales del núcleo de Java, como las del paquete `java.lang` (por ejemplo, `Object`, `String`), que se encuentran en el archivo `rt.jar` en versiones antiguas de Java o en los módulos del sistema en Java 9 y posteriores.
- **Extension Class Loader (Platform Class Loader en Java 9+):** Es hijo del Bootstrap Class Loader. Carga las clases de las extensiones del JDK, que se encuentran en el directorio `jre/lib/ext` o en las rutas especificadas por la propiedad del sistema `java.ext.dirs`.
- **Application Class Loader (System Class Loader):** Es hijo del Extension Class Loader y es el cargador por defecto para las clases de la aplicación. Carga las clases que se encuentran en la ruta especificada por la variable de entorno `CLASSPATH` o por las opciones `-cp` o `-classpath` en la línea de comandos.

Estos cargadores operan bajo un principio crucial conocido como el **Modelo de Delegación (Parent-First)**. Cuando se solicita a un cargador de clases (por ejemplo, el Application Class Loader) que cargue una clase, este no intenta cargarla inmediatamente. En su lugar, primero delega la solicitud a su cargador padre (el Extension Class Loader). El padre, a su vez, delega la solicitud a su propio padre (el Bootstrap Class Loader). La búsqueda de la clase se realiza de arriba hacia abajo en la jerarquía. Solo si el Bootstrap Class Loader no puede encontrar la

clase, el Extension Class Loader intenta cargarla. Y solo si este también falla, el Application Class Loader original intenta encontrar y cargar la clase por sí mismo.

Este modelo de delegación no es simplemente una estrategia de búsqueda; es un pilar fundamental de la seguridad y la estabilidad de Java. Garantiza que las clases del núcleo de la plataforma, cargadas por el Bootstrap Class Loader, nunca puedan ser reemplazadas o suplantadas por clases con el mismo nombre provenientes del classpath de la aplicación. Por ejemplo, una aplicación maliciosa no puede proporcionar su propia versión de `java.lang.String` para violar las reglas de seguridad, porque la solicitud de carga de `java.lang.String` siempre será satisfecha por el Bootstrap Class Loader antes de que cualquier otro cargador tenga la oportunidad de actuar. Este mecanismo crea un "sandbox" a nivel de carga de clases, previniendo la inestabilidad y las vulnerabilidades de seguridad de manera proactiva.

La Geografía de la Memoria: Un Recorrido por las Áreas de Datos en Tiempo de Ejecución (Runtime Data Areas)

Para ejecutar un programa, la JVM organiza la memoria que necesita en varias áreas de datos en tiempo de ejecución. Cada área tiene un propósito específico y un ciclo de vida definido. Estas áreas se pueden clasificar en dos grupos: las que son compartidas por todos los hilos de la aplicación y las que son privadas para cada hilo individual.

Áreas de Memoria Compartidas

- **Heap:** Es el área de memoria principal y más grande de la JVM. Todos los objetos (instancias de clases) y arrays creados durante la ejecución del programa con el operador `new` se asignan en el Heap. El Heap se crea al iniciar la JVM y es compartido por todos los hilos, lo que significa que los objetos en el Heap pueden ser accedidos y modificados por múltiples hilos (requiriendo mecanismos de sincronización para garantizar la seguridad). La gestión de esta área es automática y está a cargo del Recolector de Basura. Si una aplicación intenta crear un objeto y no hay suficiente espacio disponible en el Heap, la JVM lanzará un error `java.lang.OutOfMemoryError`.
- **Method Area (Área de Métodos):** Esta área almacena información por clase, no por instancia. Contiene la estructura de cada clase cargada, como el nombre de la clase, su superclase, los nombres y descriptores de sus campos y métodos, el bytecode de los métodos y constructores, y las variables estáticas. También contiene una estructura por clase llamada **Runtime Constant Pool**, que almacena constantes y referencias simbólicas. Al igual que el Heap, se crea al iniciar la JVM y es compartida por todos los hilos. En las versiones de Java anteriores a la 8, una implementación común de esta área era el "Permanent Generation" (PermGen), que tenía un tamaño fijo y podía causar errores de `OutOfMemoryError: PermGen space`. Desde Java 8, el PermGen fue reemplazado por el **Metaspace**, que se asigna en la memoria nativa del sistema operativo en lugar del Heap de la JVM, lo que lo hace más flexible y reduce la probabilidad de este tipo de error.

Áreas de Memoria por Hilo

- **JVM Stack (Pila):** Cada hilo de ejecución en una aplicación Java tiene su propia pila privada, que se crea al mismo tiempo que el hilo. La pila almacena **frames** (marcos de

pila). Se crea un nuevo frame cada vez que se invoca un método, y se destruye cuando el método completa su ejecución (ya sea retornando un valor o lanzando una excepción). Cada frame contiene:

- **Variables Locales:** Un array que almacena las variables locales y los parámetros del método.
- **Pila de Operandos:** Un espacio de trabajo LIFO (Last-In, First-Out) que las instrucciones de bytecode utilizan para realizar cálculos intermedios.
- **Datos del Frame:** Información de soporte para el método, como una referencia al Runtime Constant Pool de la clase. Dado que cada hilo tiene su propia pila, no se necesita sincronización para acceder a las variables locales. Si la recursión de un método es demasiado profunda o se crean demasiados frames, la pila puede desbordarse, lo que resulta en un error `java.lang.StackOverflowError`.
- **PC Register (Registro Contador de Programa):** Cada hilo también tiene su propio registro PC. Este registro es muy simple y almacena la dirección de la instrucción de bytecode de la JVM que se está ejecutando actualmente. A medida que el hilo ejecuta instrucciones, el registro PC se actualiza para apuntar a la siguiente instrucción. Si el método que se está ejecutando es un método nativo (no escrito en Java), el valor del registro PC es indefinido.
- **Native Method Stack (Pila de Métodos Nativos):** Es similar a la JVM Stack, pero está dedicada a la ejecución de métodos nativos, que son métodos escritos en otros lenguajes como C o C++. Si la JVM de una plataforma no soporta métodos nativos, puede que no proporcione esta pila. Al igual que la JVM Stack, es privada para cada hilo.

El Corazón Operativo: El Motor de Ejecución (Execution Engine)

El Motor de Ejecución es el componente central que lee el bytecode de las áreas de memoria y lo ejecuta. Es el verdadero procesador de la JVM. Para ejecutar el bytecode, utiliza varios componentes clave.

Intérprete

El intérprete lee el bytecode instrucción por instrucción, la interpreta y la ejecuta secuencialmente. Este enfoque es simple y garantiza la portabilidad, ya que el mismo intérprete puede ejecutarse en cualquier plataforma. Sin embargo, la interpretación puede ser lenta, especialmente para código que se ejecuta repetidamente (como en un bucle), porque cada instrucción debe ser reinterpretada cada vez que se encuentra.

Compilador Just-In-Time (JIT)

Para superar la desventaja de rendimiento del intérprete, la JVM moderna incorpora un Compilador Just-In-Time (JIT). El motor de ejecución incluye un **Profiler**, un componente que monitoriza el código mientras se ejecuta para identificar "hotspots" —métodos o bucles que se ejecutan con mucha frecuencia—. Cuando se identifica un hotspot, el compilador JIT entra en acción. En lugar de seguir interpretando ese código, el JIT lo compila en tiempo de ejecución a código máquina nativo, que es específico de la plataforma subyacente y se ejecuta mucho más rápido. Este código nativo compilado se almacena en una caché. Las futuras llamadas a ese hotspot ejecutarán directamente el código nativo, evitando por completo la sobrecarga de la interpretación. Este enfoque adaptativo permite que Java logre un rendimiento que, en muchos

casos, se acerca o incluso supera al de los lenguajes compilados estáticamente como C++.

Recolector de Basura (Garbage Collector - GC)

El Recolector de Basura es un programa que se ejecuta en segundo plano y es responsable de la gestión automática de la memoria en el Heap. Su trabajo es encontrar objetos que ya no están en uso por la aplicación y liberar la memoria que ocupan.

- **Proceso Básico (Mark-Sweep-Compact):** La mayoría de los recolectores de basura siguen un proceso fundamental de tres pasos:
 1. **Marking (Marcado):** El GC comienza desde un conjunto de "raíces de recolección" (GC Roots), que son referencias a objetos que siempre son accesibles (como variables locales en hilos activos, variables estáticas, etc.). Recorre el grafo de objetos en el Heap, marcando todos los objetos que son alcanzables desde estas raíces como "vivos".
 2. **Sweeping (Barrido):** Después del marcado, el GC recorre todo el Heap. Cualquier objeto que no haya sido marcado como "vivo" se considera basura y la memoria que ocupa se reclama.
 3. **Compacting (Compactación):** Después de liberar la memoria, el Heap puede quedar fragmentado, con pequeños bloques de memoria libre dispersos entre los objetos vivos. Para evitar esto, el GC puede realizar una fase de compactación, que consiste en mover todos los objetos vivos juntos a un extremo del Heap. Esto deja un gran bloque contiguo de memoria libre, lo que hace que las futuras asignaciones de objetos sean mucho más rápidas y sencillas.
- **Recolección Generacional:** Para optimizar el proceso, la mayoría de las JVM modernas utilizan una estrategia de recolección generacional. Se basa en la "hipótesis generacional débil": la mayoría de los objetos creados en una aplicación tienen una vida muy corta. Por lo tanto, el Heap se divide en varias "generaciones":
 - **Young Generation (Generación Joven):** Aquí es donde se asignan inicialmente todos los objetos nuevos. Esta generación se recolecta con frecuencia a través de eventos llamados **Minor GC**. Como la mayoría de los objetos mueren jóvenes, estas recolecciones son muy eficientes y rápidas. Los objetos que sobreviven a varias recolecciones menores son "promovidos" a la siguiente generación.
 - **Old Generation (Generación Vieja) o Tenured Space:** Aquí residen los objetos de larga vida que han sobrevivido a la Young Generation. Esta área se recolecta con menos frecuencia, a través de eventos más costosos llamados **Major GC** o **Full GC**, que a menudo implican pausar toda la aplicación ("stop-the-world").

La evolución de la JVM ha sido un reflejo directo de la evolución de los casos de uso de Java. Las primeras versiones, adecuadas para applets simples, dependían en gran medida de la interpretación y eran notoriamente lentas. La introducción del compilador JIT y la máquina virtual HotSpot fue una respuesta directa a la necesidad de un alto rendimiento en aplicaciones de servidor. De manera similar, la evolución de los algoritmos del recolector de basura refleja las cambiantes demandas del mercado. Los primeros recolectores se centraban en el *throughput* (rendimiento total), ideal para tareas de procesamiento por lotes. Sin embargo, el auge de las aplicaciones interactivas, los microservicios y los sistemas de baja latencia, que no pueden tolerar largas pausas de "stop-the-world", impulsó el desarrollo de recolectores de basura concurrentes y de baja latencia como G1GC y ZGC. Cada avance técnico en el Motor de Ejecución ha sido causado por la necesidad de que la plataforma Java siga siendo relevante

y competitiva en los dominios de aplicación dominantes de cada era tecnológica.

Tabla 2: Comparativa de los Principales Algoritmos de Recolección de Basura en la JVM

Nombre del Colector	Característica Principal	Modelo de Hilos	Caso de Uso Ideal	Argumento JVM
Serial GC	El más simple, "stop-the-world".	Monohilo	Aplicaciones cliente pequeñas, entornos con un solo núcleo.	-XX:+UseSerialGC
Parallel GC	Optimizado para alto <i>throughput</i> , "stop-the-world".	Multihilo	Aplicaciones intensivas en CPU, procesamiento por lotes. Por defecto hasta Java 8.	-XX:+UseParallelGC
G1 (Garbage-First) GC	Divide el Heap en regiones, pausas predecibles.	Multihilo, concurrente	Grandes heaps (>4GB), aplicaciones sensibles a la latencia. Por defecto desde Java 9.	-XX:+UseG1GC
ZGC (Z Garbage Collector)	Pausas extremadamente bajas (<10ms), concurrente.	Multihilo, concurrente	Aplicaciones de muy baja latencia, heaps masivos (multi-terabyte).	-XX:+UseZGC

Fuentes:

Parte IV: Evolución, Ecosistema Moderno y Perspectivas Futuras

Tras establecer sus fundamentos y diseccionar su arquitectura interna, es crucial situar a Java en su contexto contemporáneo. La plataforma no ha permanecido estática; ha evolucionado bajo una nueva administración, ha adoptado un modelo de desarrollo de código abierto y ha establecido un ciclo de vida de versiones que equilibra la innovación con la estabilidad empresarial. Además, su componente más valioso, la JVM, ha trascendido el propio lenguaje Java para convertirse en una plataforma políglota de alto rendimiento. Esta sección final analiza la trayectoria moderna de Java y sus perspectivas de futuro.

De Sun a Oracle: Hitos Clave, la Creación de OpenJDK y el Panorama Actual de las Distribuciones

Un momento decisivo en la historia de Java ocurrió en 2009, cuando Oracle Corporation completó la adquisición de Sun Microsystems, transfiriendo así la propiedad y la administración

de la plataforma Java a uno de los gigantes del software empresarial. Esta adquisición generó tanto expectación como incertidumbre en la comunidad.

Unos años antes, en 2006-2007, Sun Microsystems había dado un paso fundamental al liberar la mayor parte del código fuente de Java bajo la licencia GNU General Public License (GPL), creando el proyecto **OpenJDK**. Desde la versión 7 de Java SE, OpenJDK se ha convertido en la implementación de referencia oficial de la plataforma Java. Esto significa que el desarrollo de nuevas características y la evolución del lenguaje y la JVM ocurren principalmente dentro del proyecto OpenJDK, una comunidad de código abierto en la que contribuyen Oracle, Red Hat, IBM, Google, SAP y muchos otros actores de la industria, así como desarrolladores individuales.

El **Oracle JDK**, la distribución comercial de Oracle, se construye a partir del mismo código fuente de OpenJDK. Históricamente, Oracle JDK incluía algunas características de código cerrado adicionales, como Java Flight Recorder y Mission Control, pero con el tiempo, estas también se han donado a OpenJDK y ahora están disponibles en ambas distribuciones. Hoy en día, la diferencia técnica entre Oracle JDK y OpenJDK es prácticamente nula. Las distinciones clave radican en el licenciamiento y el soporte :

- **Licenciamiento:** OpenJDK es completamente gratuito y de código abierto bajo la licencia GPLv2 con una excepción de enlace. Por otro lado, aunque Oracle ofrece versiones de su JDK bajo una licencia "No-Fee Terms and Conditions" (NFTC) para uso gratuito en producción, su uso a largo plazo y el acceso a actualizaciones para versiones más antiguas requieren una suscripción comercial de pago, un cambio de política que se implementó en 2019.
- **Soporte:** El soporte para OpenJDK proviene de la comunidad y de varios proveedores comerciales. Oracle ofrece soporte comercial exclusivamente para su Oracle JDK.

La apertura de OpenJDK ha fomentado un ecosistema vibrante de distribuciones. Varias organizaciones toman el código fuente de OpenJDK, lo compilan y ofrecen sus propias versiones, a menudo con soporte comercial a largo plazo. Entre las distribuciones más populares se encuentran **Red Hat OpenJDK**, **Azul Zulu**, **Amazon Corretto**, **Eclipse Temurin** y **Microsoft Build of OpenJDK**. Esta diversidad ofrece a las empresas flexibilidad y opciones para elegir un proveedor de JDK que se ajuste a sus necesidades técnicas y comerciales.

Cadencia y Estabilidad: El Modelo de Versiones de Soporte a Largo Plazo (LTS)

Históricamente, las nuevas versiones de Java se lanzaban cada varios años, impulsadas por la finalización de un conjunto de características importantes. A partir de Java 9, la plataforma adoptó un modelo de lanzamiento basado en el tiempo, mucho más predecible: una nueva versión se publica cada seis meses, en marzo y septiembre.

Este rápido ciclo de innovación planteaba un desafío para las grandes empresas, que valoran la estabilidad y no pueden permitirse migrar sus sistemas críticos cada seis meses. Para abordar esta tensión entre la innovación rápida y la estabilidad empresarial, se introdujo el concepto de versiones de **Soporte a Largo Plazo (Long-Term Support - LTS)**.

- **Versiones no-LTS (Feature Releases):** Son las versiones que se publican cada seis meses. Reciben actualizaciones y soporte solo durante seis meses, hasta que se publica la siguiente versión. Actúan como un vehículo para la innovación continua, permitiendo que nuevas características se introduzcan y se prueben en el ecosistema.
- **Versiones LTS:** Cada dos años (inicialmente cada tres), una de las versiones se designa

como LTS. Estas versiones reciben soporte extendido durante muchos años (típicamente 8 años o más de soporte comercial por parte de proveedores como Oracle o Azul) con actualizaciones trimestrales de seguridad y corrección de errores.

Este modelo dual es una solución sofisticada a las necesidades conflictivas de dos comunidades. La rápida sucesión de versiones no-LTS causa que la innovación sea constante, satisfaciendo a los desarrolladores ávidos de nuevas herramientas. La existencia de versiones LTS causa que la adopción empresarial sea segura y manejable, proporcionando puntos de anclaje estables. Las versiones LTS, como **Java 8, 11, 17, 21 y la futura 25**, son el estándar de facto en entornos de producción, ya que ofrecen la predictibilidad y la longevidad que las empresas requieren. Encuestas de la industria confirman que una gran mayoría de las aplicaciones en producción se ejecutan sobre estas versiones LTS.

Más Allá de Java: La JVM como una Plataforma Políglota

La brillante decisión arquitectónica de separar el lenguaje de su entorno de ejecución ha tenido una consecuencia de largo alcance: la JVM ha trascendido el propio lenguaje Java. Cualquier lenguaje de programación que pueda ser compilado al formato de bytecode de la JVM puede ejecutarse en ella, aprovechando instantáneamente décadas de optimización de rendimiento, gestión de memoria robusta, seguridad y un vasto ecosistema de bibliotecas y herramientas. Esto ha dado lugar a un próspero ecosistema de "lenguajes JVM" que ofrecen paradigmas y sintaxis alternativos a Java, pero que comparten el mismo tiempo de ejecución de alto rendimiento.

Caso de Estudio: Kotlin y Scala

Dos de los lenguajes JVM alternativos más exitosos son Kotlin y Scala.

- **Scala**, lanzado en 2004, fue uno de los pioneros. Combina la programación orientada a objetos con un fuerte enfoque en la programación funcional, ofreciendo una sintaxis expresiva y potente para construir sistemas concurrentes y escalables.
- **Kotlin**, desarrollado por JetBrains y lanzado en 2012, fue diseñado como un "mejor Java". Ofrece una sintaxis más concisa, seguridad contra referencias nulas integrada en el sistema de tipos y excelentes herramientas. Su característica más destacada es su perfecta interoperabilidad con Java: el código Kotlin puede llamar a código Java y viceversa sin problemas, permitiendo una adopción gradual en proyectos existentes.

Google lo designó como el lenguaje preferido para el desarrollo de aplicaciones Android. El proceso de ejecución para estos lenguajes es el mismo: el código fuente (por ejemplo, en un archivo .kt para Kotlin) se compila con su propio compilador (ej. kotlinc) a archivos .class que contienen bytecode estándar de la JVM. Estos archivos .class pueden ser empaquetados en un archivo JAR y ejecutados con el comando `java -jar`, exactamente igual que una aplicación Java. El éxito de estos lenguajes demuestra que el mayor legado de Java podría no ser el lenguaje en sí, sino la propia JVM. Se ha convertido en una de las plataformas de ejecución de software más robustas y versátiles jamás creadas, una especie de "máquina virtual universal" que ahora sirve como un runtime para una nueva generación de lenguajes innovadores.

Innovaciones Recientes y el Futuro de la JVM: Proyectos Loom, Valhalla y GraalVM

La evolución de la plataforma Java está lejos de terminar. Varios proyectos de vanguardia, desarrollados dentro de OpenJDK, están sentando las bases para el futuro de la JVM:

- **Proyecto Loom:** Introduce la concurrencia ligera en la JVM a través de "hilos virtuales" (virtual threads). Estos permiten escribir código concurrente de alto rendimiento en un estilo de programación secuencial simple, facilitando la creación de aplicaciones que pueden manejar millones de tareas concurrentes con un uso de recursos muy bajo.
- **Proyecto Valhalla:** Tiene como objetivo mejorar el modelo de memoria de Java introduciendo "tipos de valor" (value types). Esto permitirá a los programadores crear agregados de datos densos en memoria, similares a los structs de C, mejorando significativamente el rendimiento al reducir la indirección de memoria y la presión sobre el recolector de basura.
- **Proyecto Panama:** Busca mejorar y simplificar la interconexión entre el código Java y el código nativo (API de C/C++), reemplazando la antigua y compleja Java Native Interface (JNI).

Además de estos proyectos, **GraalVM** representa una evolución significativa de la propia máquina virtual. GraalVM es un runtime de alto rendimiento que extiende la JVM con varias capacidades notables. Una de las más importantes es la compilación Ahead-Of-Time (AOT) a través de su componente **Native Image**. Esto permite compilar una aplicación Java directamente en un ejecutable nativo autónomo, lo que resulta en tiempos de arranque casi instantáneos y un consumo de memoria mucho menor, características ideales para microservicios y funciones serverless. GraalVM también es una máquina virtual políglota, capaz de ejecutar lenguajes como JavaScript, Python y Ruby junto con lenguajes JVM, todo dentro del mismo runtime.

Conclusión: El Legado y la Relevancia Continua de Java en la Era de la Nube y la IA

Desde sus humildes comienzos como un proyecto para dispositivos inteligentes hasta su dominio en el desarrollo empresarial y su actual adaptación a la era de la nube, la historia de Java es una de notable resiliencia y adaptabilidad. Su filosofía de diseño original, centrada en la portabilidad, la simplicidad, la robustez y la seguridad, demostró ser extraordinariamente premonitoria. Estos principios fundamentales permitieron a Java no solo sobrevivir, sino prosperar a través de múltiples transformaciones tecnológicas.

El análisis de su arquitectura revela un diseño ingenioso. El modelo híbrido de compilación a bytecode y ejecución en una máquina virtual fue la clave para resolver el dilema de la portabilidad sin sacrificar por completo el rendimiento. La separación de la plataforma en JDK, JRE y JVM creó un ecosistema flexible que atiende tanto a desarrolladores como a entornos de producción de manera eficiente y segura.

Sin embargo, el verdadero genio de la plataforma reside en la propia Máquina Virtual de Java. La JVM es una pieza de ingeniería de software de vanguardia que ha evolucionado continuamente. Sus sofisticados compiladores Just-In-Time transforman a Java en un lenguaje de alto rendimiento, mientras que sus avanzados recolectores de basura gestionan la memoria de forma autónoma, permitiendo a los desarrolladores centrarse en la lógica de negocio. La evolución de la JVM para soportar un ecosistema de lenguajes políglota y la aparición de innovaciones como los hilos virtuales del Proyecto Loom y la compilación nativa con GraalVM demuestran que la plataforma está lejos de ser estática.

Lejos de ser un lenguaje obsoleto, Java sigue siendo una de las plataformas de desarrollo más

populares y fiables del mundo. Su robusto ecosistema, su rendimiento probado en batalla y su continua evolución la posicionan como una tecnología fundamental para enfrentar los desafíos del desarrollo nativo en la nube, el big data y la inteligencia artificial. El viaje de Java, de una taza de café a la columna vertebral de la infraestructura digital global, es un testimonio de una visión arquitectónica sólida y una capacidad incesante de adaptación.

Fuentes citadas

1. Origen e Historia de Java_ Un Recorrido Por Tres D | PDF | Java (lenguaje de programación) - Scribd, <https://es.scribd.com/document/921050289/Origen-e-Historia-de-Java-Un-Recorrido-Por-Tres-D>
2. Java cumple 30 años: Historia, evolución y futuro de un lenguaje universal - OpenWebinars, <https://openwebinars.net/blog/java-cumple-30-anos/>
3. Historia de Java. Una historia completa del desarrollo de Java, de 1991 a 2021 - CodeGym, <https://codegym.cc/es/groups/posts/es.594.historia-de-java-una-historia-completa-del-desarrollo-de-java-de-1991-a-2021>
4. Unidad I: 1.2 Elementos del Lenguaje 1.2.1 Historia de Java. 1.2.2 ¿Qué es Java? 1.2.3 Características de Java. 1.2.4 La Máq, <https://jmpovedar.files.wordpress.com/2014/08/introduccion-a-java.pdf>
5. Java (lenguaje de programación) - Wikipedia, la enciclopedia libre, [https://es.wikipedia.org/wiki/Java_\(lenguaje_de_programaci%C3%B3n\)](https://es.wikipedia.org/wiki/Java_(lenguaje_de_programaci%C3%B3n))
6. Historia de Java: un lenguaje de programación con recorrido - Tokio School, <https://www.tokioschool.com/noticias/historia-java-el-origen-de-este-lenguaje-de-programacion/>
7. ¿Qué es Java y cuáles son sus características? – Fiscalía General del Estado de Veracruz, <https://fiscaliaveracruz.gob.mx/que-es-java-y-cuales-son-sus-caracteristicas/>
8. Introducción a Java - Manual Web, <https://www.manualweb.net/java/introduccion-java/>
9. ¿Qué es Java? - Formatalent, <https://formatalent.com/que-es-java/>
10. Java: un lenguaje que aporta rapidez y facilidad de uso. - E-Saurio's Blog - Mobile Experts, <https://blog.e-saurio.com/java-un-lenguaje-que-aporta-rapidez-y-facilidad-de-uso/>
11. Introducción al lenguaje de programación Java - Cable Naranja, <https://www.cablenaranja.com/introduccion-al-lenguaje-de-programacion-java/>
12. Java (lenguaje de programación) - EcuRed, <https://www.ecured.cu/JAVA>
13. ¿Qué tan similares son Java y C++? : r/learnprogramming - Reddit, https://www.reddit.com/r/learnprogramming/comments/r0ssuv/how_similar_are_java_and_c/?tl=es-419
14. Java vs. C++ Comparison: What Are the Differences in These Programming Languages?, <https://www.coursera.org/articles/java-vs-c>
15. ¿Cuáles son las grandes diferencias entre Java y C++, cuál es más fácil de utilizar y aprender? - Quora, <https://es.quora.com/Cu%C3%A1les-son-las-grandes-diferencias-entre-Java-y-C-cu%C3%A1l-es-m%C3%A1s-f%C3%A1cil-de-utilizar-y-aprender>
16. Comparativa entre C++ y Java - Matias Giordanengo - Prezi, <https://prezi.com/p/q2uioktasmmi/comparativa-entre-c-y-java/>
17. Comparison of Java and C++ - Wikipedia, https://en.wikipedia.org/wiki/Comparison_of_Java_and_C%2B%2B
18. Todavía no entiendo qué beneficios ofrece Java sobre C++ hoy en día, excepto la "seguridad de memoria" y un framework web como Spring. No es más fácil de escribir o mantener y es mucho más lento. Si alguien entiende la POO y los patrones de diseño, ¿por qué no usar C++ en primer lugar? : - Reddit, https://www.reddit.com/r/cpp/comments/w2wt8r/i_still_dont_get_what_benefits_does_java_provide/?tl=es-es
19. JDK vs JRE vs JVM: Difference between them - BeginnersBook, <https://beginnersbook.com/2022/06/jdk-vs-jre-vs-jvm-difference-between-them/>
20. JDK vs JRE

vs JVM in Java: Key Differences Explained - DigitalOcean, <https://www.digitalocean.com/community/tutorials/difference-jdk-vs-jre-vs-jvm> 21. JVM vs. JRE vs. JDK: What's the Difference? - IBM, <https://www.ibm.com/think/topics/jvm-vs-jre-vs-jdk> 22. Curso Gratis de Java para Hackers – Máquina virtual Java (JVM) - Álvaro Chirou, <https://achirou.com/curso-gratis-de-java-para-hackers-maquina-virtual-java-jvm/> 23. Comprensión de la arquitectura de la máquina virtual Java (JVM) - AppMaster, <https://appmaster.io/es/blog/arquitectura-de-maquina-virtual-java> 24. JDK vs JRE vs JVM: Key Differences (Updated 2025) - igmGuru, <https://www.igmguru.com/blog/differences-jdk-jre-jvm> 25. ¿Qué es el entorno de ejecución de Java (JRE)? - Amazon AWS, <https://aws.amazon.com/es/what-is/java-runtime-environment/> 26. Compilación y ejecución de programas en JAVA. 1. Introducción. 2. Compilación. - UNAM, <http://profesores.fi-b.unam.mx/laloea/pamn/htm/docs/java.pdf> 27. Cómo compilar y ejecutar una aplicación Java desde la línea de comandos - LabEx, <https://labex.io/es/tutorials/java-how-to-compile-and-run-a-java-application-from-the-command-line-413959> 28. Proceso de ejecución en Java. [English Version] | by Daniel García Solla | Medium, <https://cardstdani.medium.com/proceso-de-ejecuci%C3%B3n-en-java-3a8c96e557f7> 29. Course Módulo 1. Java Syntax - Lecture: Compilando una clase en bytecode desde la consola - CodeGym, <https://codegym.cc/es/quests/lectures/es.cgu.module1.lecture10> 30. Conceptos básicos sobre compilación y compiladorespaña | Lenovo España, <https://www.lenovo.com/es/es/glossary/compilation/> 31. JVM Architecture — How Java Really Works - YouTube, <https://www.youtube.com/watch?v=QfOHeFjUOwg> 32. Compilación y ejecución en programación [Guía 2025] - KeepCoding, <https://keepcoding.io/blog/principios-de-compilacion-y-ejecucion/> 33. Bytecode Java - Wikipedia, la enciclopedia libre, https://es.wikipedia.org/wiki/Bytecode_Java 34. Fuentes, Bytecode, Depuración, <https://flounder.dev/es/posts/sources-bytecode-debugging/> 35. What Is Java Class Loader? - ITU Online IT Training, <https://www.ituonline.com/tech-definitions/what-is-java-class-loader/> 36. Java class loader - Wikipedia, https://en.wikipedia.org/wiki/Java_class_loader 37. Class Loaders in Java - Baeldung, <https://www.baeldung.com/java-classloaders> 38. The Java Virtual Machine - The Class Loader Subsystem - Artima, <https://www.artima.com/insidejvm/ed2/jvm4.html> 39. JVM Architecture And Class Loaders Java - JavaTechOnline, <https://javatechonline.com/jvm-architecture-and-class-loaders-java/> 40. Tutorial JVM - La arquitectura de la máquina virtual de Java explicada para principiantes, <https://www.freecodecamp.org/espanol/news/tutorial-jvm-la-arquitectura-de-la-maquina-virtual-de-java-explicada-para-principiantes/> 41. ClassLoader in Java - GeeksforGeeks, <https://www.geeksforgeeks.org/java/classloader-in-java/> 42. Máquina Virtual de Java (JVM): Componentes, arquitectura y funcionalidades avanzadas, <https://www.godaddy.com/resources/latam/tecnologia/jvm-que-como-funciona-maquina-virtual> 43. Understanding Run-Time Data Areas in Java Virtual Machine - Roy Tutorials, <https://roytuts.com/understanding-run-time-data-areas-in-java-virtual-machine/> 44. How Many Types of Memory Areas are Allocated by JVM? - GeeksforGeeks, <https://www.geeksforgeeks.org/java/how-many-types-of-memory-areas-are-allocated-by-jvm/> 45. The JVM Run-Time Data Areas | Baeldung, <https://www.baeldung.com/java-jvm-run-time-data-areas> 46. Chapter 2. The Structure of the Java Virtual Machine, <https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-2.html> 47. Runtime Data Areas Of Java - HackerEarth, <https://www.hackerearth.com/practice/notes/runtime-data-areas-of-java/> 48. Full explanation of JVM - Runtime Data Area and How JVM using it - NashTech Blog,

<https://blog.nashtechglobal.com/full-explanation-of-jvm-runtime-data-area-and-how-jvm-using-it/> 49. Execution Engine in Java - GeeksforGeeks, <https://www.geeksforgeeks.org/java/execution-engine-in-java/> 50. Understanding JVM Garbage Collection - Part 1 - A Bias For Action, <https://abiasforaction.net/understanding-jvm-garbage-collection-part-1/> 51. ¿Qué es una Máquina Virtual Java (JVM)? - Lenovo, <https://www.lenovo.com/mx/es/glosario/index/> 52. Desempeño de C++ vs. Java: todo reside en la calidad del compilador (y del programador también) | ::everac99, <https://everac99.wordpress.com/2012/12/13/desempeno-de-c-vs-java-todo-reside-en-la-calidad-del-compilador-y-del-programador-tambien/> 53. Java garbage collection: What is it and how does it work? - New Relic, <https://newrelic.com/blog/best-practices/java-garbage-collection> 54. JVM Garbage Collectors | Baeldung, <https://www.baeldung.com/jvm-garbage-collectors> 55. Java Garbage Collection Basics - Oracle, <https://www.oracle.com/webfolder/technetwork/tutorials/obe/java/gc01/index.html> 56. Java Memory Management Explained | DigitalOcean, <https://www.digitalocean.com/community/tutorials/java-jvm-memory-model-memory-management-in-java> 57. Uso de ZGC de alto rendimiento en Minecraft - Reddit, https://www.reddit.com/r/Minecraft/comments/15jsi04/performant_zgc_usage_in_minecraft/?tl=es-es 58. Differences Between Oracle JDK and OpenJDK - Baeldung, <https://www.baeldung.com/oracle-jdk-vs-openjdk> 59. Comparación entre OpenJDK y Oracle JDK - Red Hat, <https://www.redhat.com/es/topics/application-modernization/openjdk-vs-oracle-jdk> 60. Oracle JDK vs OpenJDK: Which One is Right for Your Company? - Spinnaker Support, <https://www.spinnakersupport.com/blog/2024/05/01/oracle-jdk-vs-openjdk/> 61. Oracle Java SE Support Roadmap, <https://www.oracle.com/latam/java/technologies/java-se-support-roadmap.html> 62. Differences Between OpenJDK vs Oracle JDK - Azul Systems, <https://www.azul.com/products/core/jdk-comparison-matrix/> 63. What is Java LTS and Why Does It Matter? - JRebel, <https://www.jrebel.com/blog/java-lts> 64. What does LTS mean for a Java release? - Stack Overflow, <https://stackoverflow.com/questions/76782885/what-does-lts-mean-for-a-java-release> 65. Java Support Roadmap & Lifecycle - Azul Systems, <https://www.azul.com/products/azul-support-roadmap/> 66. Java version history - Wikipedia, https://en.wikipedia.org/wiki/Java_version_history 67. Qué es una máquina virtual Java (JVM) y cómo funciona? - Lenovo, <https://www.lenovo.com/es/es/glossary/jvm/> 68. https://www.reddit.com/r/scala/comments/15dohbl/scala_or_kotlin/?tl=es#:~:text=Tanto%20Kotlin%20como%20Scala%20son,es%20m%C3%A1s%20similar%20a%20Java. 69. Comparación de dos lenguajes de programación, Scala y Kotlin y resumir qué lenguaje será más adecuado para los desarrolladores de Java, <http://www.prosenic.com/comparison-of-two-programming-languages-scala-and-kotlin-and-summarize-which-language-will-be-more-suitable-for-java-developers-5.html> 70. Kotlin vs Scala: Which JVM Language To Choose For Your Project? | SPEC INDIA, <https://www.spec-india.com/blog/kotlin-vs-scala> 71. Lenguajes JVM: ahora hablemos de Kotlin - Alquimista de código, <http://alquimistadecodigo.blogspot.com/2024/11/lenguajes-jvm-ahora-hablemos-de-kotlin.html> 72. Software Java | Oracle América Latina, <https://www.oracle.com/latam/java/>